

# Algorithm and Learning for Protein Science

Explainable AI for Trees: From Local Explanations to Global Understanding

S.M. Lundberg et al. (2020)

Guillaume Vindry

`guillaume.vindry@gmail.com`

ENS Paris-Saclay

Corentin Pernot

`corentin.pernot@ensae.fr`

ENS Paris-Saclay

March 24, 2025

- 1 Introduction
- 2 Limitations of Existing Methods
- 3 Contributions of the Paper
- 4 Methodology
- 5 Results and Applications
- 6 Discussion and Perspectives
- 7 Conclusion

## 1 Introduction

## 2 Limitations of Existing Methods

## 3 Contributions of the Paper

## 4 Methodology

## 5 Results and Applications

## 6 Discussion and Perspectives

## 7 Conclusion

# Context and Motivation

## Why Explainability for Tree-Based Models?

- Tree models (Random Forests, Gradient Boosting) are widely used.
- Accuracy alone is not enough; interpretability is crucial.
- Medicine, Biology, Finance, Public Health applications.

## Challenges in Explaining Tree-Based Models

- Strong focus on global explanations; local explanations often overlooked.
- Traditional methods may lack consistency or efficiency.
- Need for a method that balances accuracy, interpretability, and speed.

(A) "Black box" model prediction TreeExplainer → "White box" local explanation

**Figure 1:** Figure 1A of the paper, *A local explanation based on assigning a numeric measure of credit to each input feature*

- 1 Introduction
- 2 Limitations of Existing Methods**
- 3 Contributions of the Paper
- 4 Methodology
- 5 Results and Applications
- 6 Discussion and Perspectives
- 7 Conclusion

# Current Explainability Approaches for tree-based models

## Three main techniques:

- ① **Decision Path Reporting:** Not useful for ensembles.
- ② **Model-agnostic Methods (LIME, SHAP Approximation):** Computationally expensive.
- ③ **Heuristic Methods (e.g., Saabas):** Biased, inconsistent.





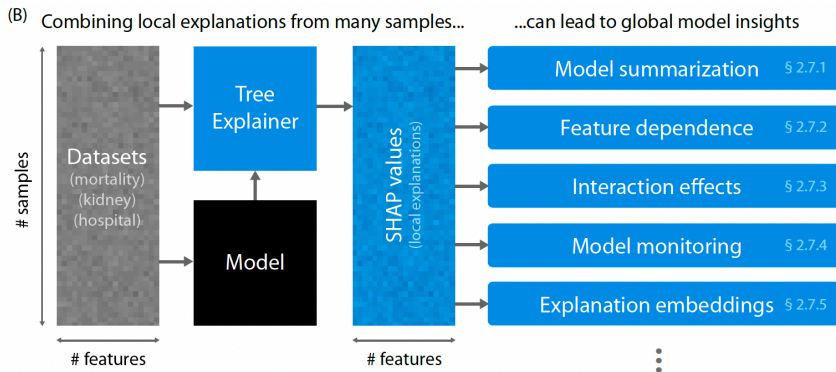


# Key Contributions

## TreeExplainer: A New Explainability Method

- Efficiently computes **SHAP values** for tree models.
- Provides **exact local explanations** in polynomial time.
- Captures **feature interaction effects**.
- Aggregates local explanations to obtain **global model insights**.

# Overview



**Figure 2:** Figure 1B of the paper, *By combining many local explanations we can represent global structure while retaining local faithfulness to the original model*

- 1 Introduction
- 2 Limitations of Existing Methods
- 3 Contributions of the Paper
- 4 Methodology**
- 5 Results and Applications
- 6 Discussion and Perspectives
- 7 Conclusion

# Shapley Values and SHAP

# Shapley Values: Theoretical Justification

## Why Shapley Values?

- Derived from cooperative game theory, ensuring fair prize attribution to each player (the only distribution method that satisfies some desirable properties).
- In our case : distributes a model's prediction  $f(x)$  among its input features.

**Definition:** Given a function  $f: 2^M \rightarrow \mathbb{R}$  (where  $M$  is the number of features), the **Shapley value** for feature  $i$  is given by:

$$\phi_i(f, x) = \sum_{S \subseteq \mathcal{M} \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} \left[ f(S \cup \{i\}) - f(S) \right]$$

where:

- $S$  is a subset of features excluding  $i$ .
- $f(S)$  is the model output when only features in  $S$  are considered.
- The weighting factor accounts for all possible feature orderings.

# SHAP values

- Shapley values are easy to compute when features are binary (player  $i$  is present or not)
- **SHAP (SHapley Additive exPlanations)** : "generalization" of Shapley values to continuous features.
- Conditional expectations to calculate the prediction in the absence of a feature.

$$f_x(S) = \mathbb{E}(f(X) | \text{do}(X_S = x_S))$$

- $\text{do}(X_S = x_S)$  means that we use an "interventional" conditional expectation : we set the frozen features to  $x_S$  instead of observing points where the features are  $x_S$ .

# SHAP properties

**SHAP values** satisfy the following properties

① **Local Accuracy (Efficiency):**

$$f(x) = \sum_{i=1}^M \phi_i(f, x)$$

The sum of all Shapley values equals the total model prediction.

② **Consistency (Monotonicity):** If a model changes so that feature  $i$  has **greater marginal impact**, then its SHAP value should not decrease:

$$f'(S \cup \{i\}) - f'(S) \geq f(S \cup \{i\}) - f(S) \Rightarrow \phi_i(f', x) \geq \phi_i(f, x)$$

③ **Missingness (Nullity):** If a feature  $i$  has no effect on the model output for all subsets:

$$f(S \cup \{i\}) = f(S) \quad \forall S \subseteq \mathcal{M}$$

then  $\phi_i(f, x) = 0$ .



# Tree SHAP

# Bruteforce Algorithm for SHAP Computation

## Intuition

- To obtain the SHAP value of a feature  $i$ , compute:

$$\phi_i(f, x) = \sum_{S \subseteq \mathcal{M} \setminus \{i\}} w(S) (f_x(S \cup \{i\}) - E[f(x)|S])$$

where  $w(S) = \frac{|S|!(M-|S|-1)!}{M!}$  ensures fair weighting across all subsets.

- The bruteforce algorithm explicitly computes these values by evaluating the model under all possible feature subsets.
- For each subset of features  $S$ , compute the model's expected output  $E[f(x)|S]$ .
- For ensemble models, simply sum over the trees :

$$\phi_i^{\text{ensemble}} = \frac{1}{T} \sum_{t=1}^T \phi_i^t$$

# Intuition for the brute force algorithm

On a simple tree in 2D

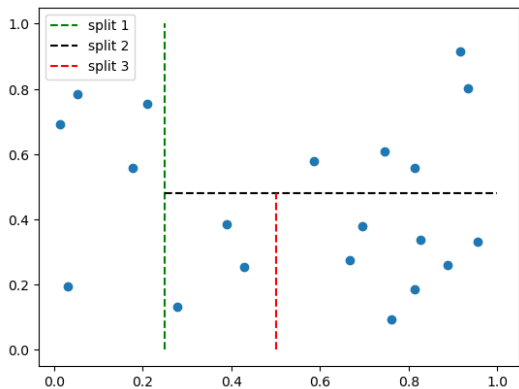


Figure 3: Simple tree with 3 splits, 2 on X and 1 on Y

# Intuition for bruteforce algorithm

Select a datapoint to be explained

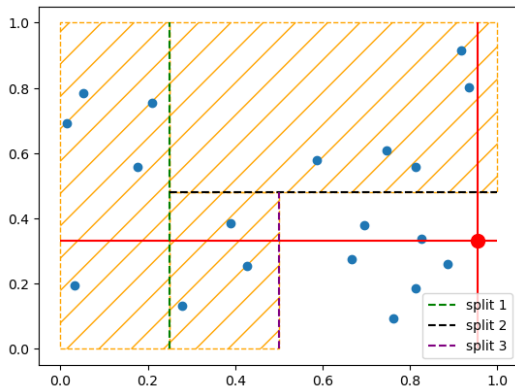


Figure 4: Features :  $S = \{X, Y\}$

# Intuition for bruteforce algorithm

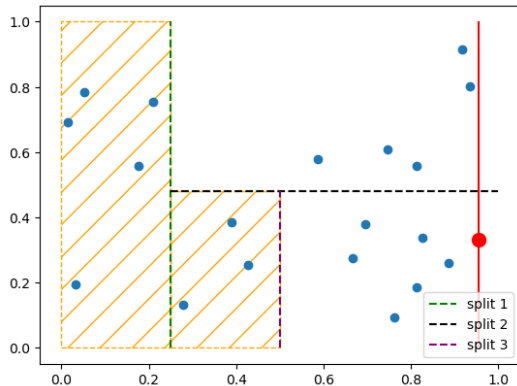


Figure 5:  $S = \{X\}$

# Intuition for bruteforce algorithm

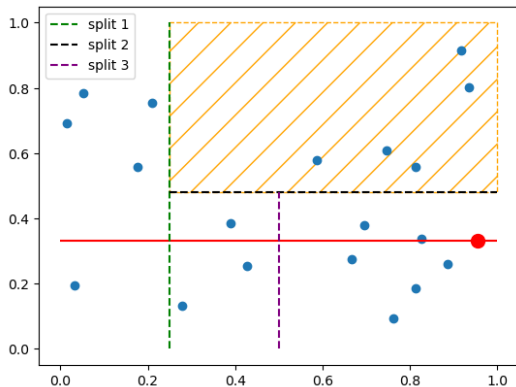


Figure 6:  $S = \{Y\}$

# Leveraging Tree Structure for Efficiency

## Computational Bottleneck

- The number of subsets  $S$  grows exponentially:  $2^M$  for  $M$  features.
- For each subset, we traverse the tree to determine  $E[f(x)|S]$ .
- This results in a total complexity of  $O(TLM2^M)$ , which is infeasible for large models.
- **Goal:** Reduce the complexity while preserving exact SHAP values.

## Key Observation

- TreeExplainer exploits the structure of trees to propagate conditional expectations at each split avoiding brute-force summation over all possible feature sets.

## Computational Improvement:

$$O(TLM2^M) \rightarrow O(TLD^2)$$

where  $D$  is the maximum tree depth, making the method feasible for large datasets,  
 $L$  is the number of leaves,  $M$  the number of features, and  $T$  the number of trees.

- 1 Introduction
- 2 Limitations of Existing Methods
- 3 Contributions of the Paper
- 4 Methodology
- 5 Results and Applications**
- 6 Discussion and Perspectives
- 7 Conclusion



# TreeExplainer vs Saabas: Consistency Matters

## Saabas problem

- Saabas values resemble SHAP but use only the tree's decision path order.
- This causes bias and violates consistency.
- Features near the root receive disproportionately low credit.

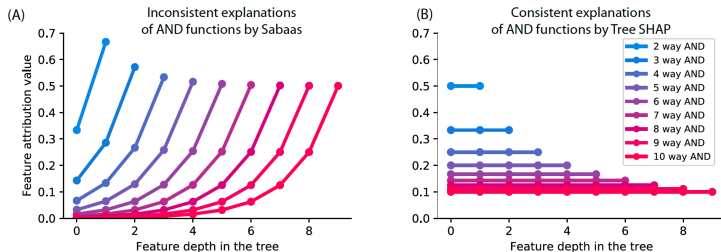
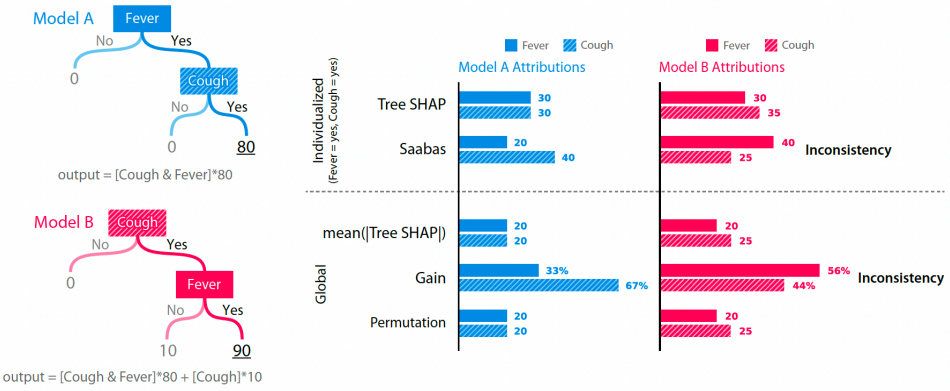


Figure 7: Supplementary Figure 1A of the paper, *TreeExplainer avoids the bias problems of Saabas*

# TreeExplainer Provides Fast Local Explanations with Guaranteed Consistency

- TreeShap avoids the inconsistency problems of current methods.

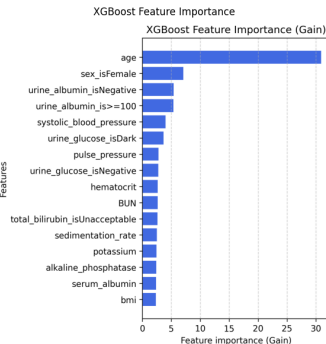
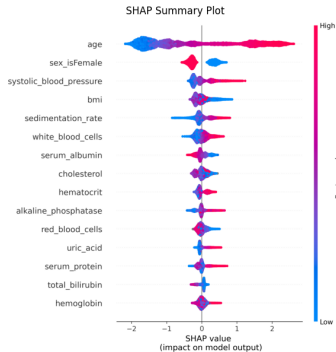
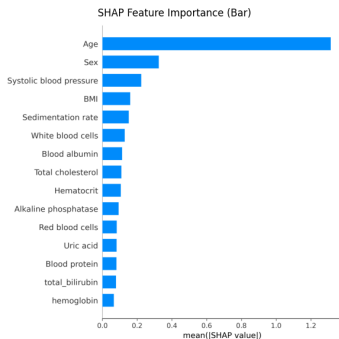


**Figure 8:** Supplementary Figure 2 of the paper, *TreeExplainer avoids the consistency problems of previous approaches*

# From Local Explanations to Global Understanding

**Key Idea:** Aggregating local SHAP explanations over all samples reveals global patterns.

- **SHAP summary plots, SHAP dependence plots:** visualize how a feature impacts predictions across the dataset, and reveal interactions.



# Extension to Measure Interaction Effects

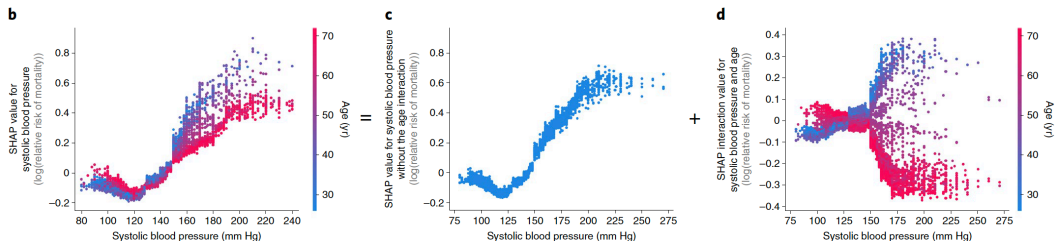
## SHAP Interaction Values

- SHAP values do not distinguish between main and interaction effects.

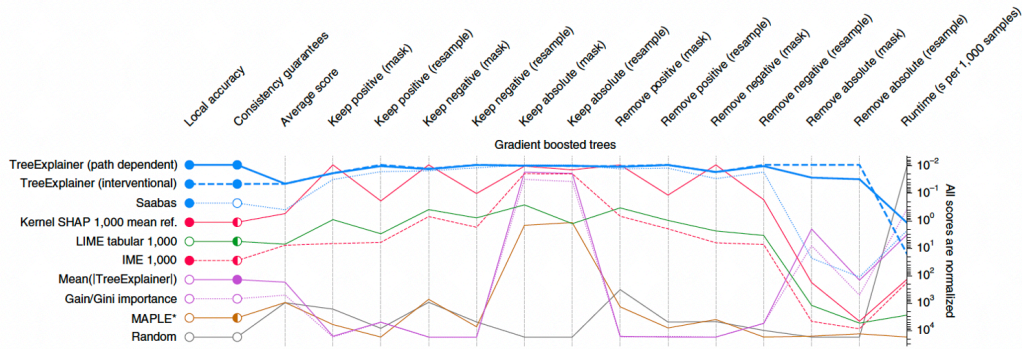
$$\text{Shap Interaction Value} = \phi_{i,i} + \phi_{i,j}$$

- Defined from the Shapley interaction index:

$$\phi_{i,j} = \frac{1}{2} \sum_{S \subseteq \mathcal{M} \setminus \{i,j\}} \frac{|S|!(M - |S| - 2)!}{(M - 1)!} \cdot [f(S \cup \{i,j\}) - f(S \cup \{i\}) - [f(S \cup \{j\}) - f(S)]]$$



# Comparison with other methods



**Figure 9:** Figure 3 of the paper, *Explanation method performance across 15 different evaluation metrics in the Chronic disease dataset*



# Future Directions

## How to improve the paper?

- **Beyond trees:** Extend efficient SHAP computations to neural networks or other complex models.
- **Higher-order interactions:** Explore triplet or multi-feature interaction effects.
- **Fast Tree SHAP**

- A set of navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.



# Takeaways

## Key contributions of the paper:

- Exact SHAP values for trees in polynomial time.
- Extension to interaction effects (SHAP interaction values).
- Aggregating local explanations → global model understanding.
- Outperforms previous methods (accuracy, consistency, speed).

Thank you!  
Questions?

## Appendix : Pseudo-code for Optimized TreeSHAP

```

18: procedure EXTEND( $m, p_z, p_o, p_i$ )
19:    $l, m = \text{len}(m), \text{copy}(m)$ 
20:    $m_{l+1}.(d, z, o, w) = (p_i, p_z, p_o, (1 \text{ if } l = 0 \text{ else } 0))$ 
21:   for  $i \leftarrow l$  to 1 do
22:      $m_{i+1}.w = m_{i+1}.w + p_o \cdot m_i.w \cdot (i/l)$ 
23:      $m_i.w = p_z \cdot m_i.w \cdot (l - i)/l$ 
24:   return  $m$ 
25: procedure UNWIND( $m, i$ )
26:    $l, n, m = \text{len}(m), m_l.w, \text{copy}(m_{1 \dots l-1})$ 
27:   for  $j \leftarrow l - 1$  to 1 do
28:     if  $m_i.o \neq 0$  then
29:        $t = m_j.w$ 
30:        $m_j.w = n \cdot l / (j \cdot m_i.o)$ 
31:        $n = t - m_j.w \cdot m_i.z \cdot (l - j) / l$ 
32:     else
33:        $m_j.w = (m_j.w \cdot l) / (m_i.z(l - j))$ 
34:   for  $j \leftarrow i$  to  $l - 1$  do
35:      $m_j.(d, z, o) = m_{j+1}.(d, z, o)$ 
36:   return  $m$ 

```

▷ Init subsets of size  $l$   
 ▷ Grow subsets using  $p_z$  and  $p_o$   
 ▷ Subsets that grow by one  
 ▷ Subsets that stay the same size  
 ▷ Return the new extended subset path  
 ▷ The inverse of the  $i$ th call to EXTEND( $m, \dots$ )  
 ▷ Shrink subsets using  $m_i.z$  and  $m_i.o$

**Algorithm 2** Tree SHAP

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

# Bibliographie I

- [1] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee.  
*From local explanations to global understanding with explainable AI for trees.*  
Nature Machine Intelligence, 2, 5667 (2020).